DATA ENGINEERING COURSE

Object-Oriented Programming, Decorators & Maintainable Code

CLASS NOTES

Instructor: Tanishq Tyagi

Topics Covered in This Session

- ✔ Designing Systems using OOP
- ✔ Combining Classes + Functions + Exceptions
- ✔ Writing Maintainable Code
- ✔ Introduction to Decorators
- ✔ Refactor Script to OOP

✔ Add Exception Handling

✔ Create Logging Decorator

Table of Contents

1. Session Overview

In this session we explored how to design software systems using Object-Oriented Programming (OOP) principles in Python. Starting with a simple procedural script, we progressively refactored it into a clean, class-based architecture — the Student Management System. Along the way we integrated robust exception handling to prevent invalid operations, and learned how to create reusable logging decorators that wrap any function with before/after logging behaviour. These concepts collectively represent the foundation of writing production-grade, maintainable Python code.

2. Learning Objectives

By the end of this session, students will be able to:

- Define classes and instantiate objects in Python.
- Implement __init__, instance methods, and use self correctly.
- Design multi-class systems (e.g., Student + StudentManager).
- Apply exception handling using raise and ValueError.
- Understand what a decorator is and how it works under the hood.
- Write a generic logging decorator using *args and **kwargs.
- Apply the @decorator syntax to any method.
- Explain why maintainable code matters in production environments.
- Connect OOP and decorator patterns to real Data Engineering pipelines.

3. Object-Oriented Programming (OOP)

3.1 What is OOP?

Object-Oriented Programming is a programming paradigm that organises code around **objects** — real-world entities that bundle both **data (attributes)** and **behaviour (methods)** together. Instead of writing loose functions that operate on scattered variables, OOP groups related data and logic into a single, reusable unit called a **class**.

> ■ **Key Concept**
>
> A class is a blueprint. An object is a specific instance created from that blueprint.

Why OOP?

- **Encapsulation** — data and behaviour are bundled together, hiding implementation details.
- **Reusability** — write once, instantiate many times.
- **Maintainability** — changes are localised within the class.
- **Scalability** — new features are added by extending existing classes.
- **Readability** — code mirrors the real-world problem domain.

3.2 Classes and Objects

A **class** is defined using the **class** keyword. An **object** is created by calling the class like a function. Every object has its own copy of attributes, but shares the class's method definitions.

```
# Defining a class
class Student:
    pass

# Creating objects (instances)
student1 = Student()
student2 = Student()

# Each object is independent
print(student1 is student2)   # Output: False
```

3.3 The __init__ Constructor

The **__init__** method is called automatically when a new object is created. It initialises the object's attributes. The first parameter is always **self**, which refers to the current instance being created.

```
class Student:
    def __init__(self, name, roll_no, marks):
        # Validate that name is not empty
        if not name:
```

```
            raise ValueError('Name cannot be empty!')
        self.name    = name     # instance attribute
        self.roll_no = roll_no  # instance attribute
        self.marks   = marks    # instance attribute

# Creating a Student object
std1 = Student('Vibhore', 21, 99)
print(std1.name)     # Output: Vibhore
print(std1.roll_no)  # Output: 21
```

> ■ **Note**
>
> • self is NOT a keyword — it is a convention. Python passes the instance automatically as the first argument.

3.4 Instance Methods

Instance methods are functions defined inside a class that operate on the object's data via **self**. They can read attributes, modify attributes, and call other methods.

```
class Student:
    def __init__(self, name, roll_no, marks):
        if not name:
            raise ValueError('Name cannot be empty!')
        self.name    = name
        self.roll_no = roll_no
        self.marks   = marks

    def calculate_grade(self):
        if self.marks >= 90:
            return 'A'
        elif self.marks >= 75:
            return 'B'
        elif self.marks >= 60:
            return 'C'
        else:
            return 'D'

    def display_details(self):
        print(f'Name:   {self.name}')
        print(f'Roll:   {self.roll_no}')
        print(f'Marks:  {self.marks}')
        print(f'Grade:  {self.calculate_grade()}')

# Usage
```

```
std = Student('Vibhore', 21, 99)
std.display_details()
```

Expected Output:

```
Name:   Vibhore
Roll:   21
Marks:  99
Grade:  A
```

Line-by-line explanation:

- calculate_grade(self) — reads self.marks and returns a letter grade.
- display_details(self) — prints all attributes and calls calculate_grade() internally.
- self.calculate_grade() — calling one method from another via self.

3.5 Designing the Student Class

The Student class represents a single student entity. Its responsibilities are limited to storing that student's data and providing behaviour specific to one student (calculating grade, displaying details). This follows the **Single Responsibility Principle**.

3.6 Designing the StudentManager Class

The StudentManager class acts as a **container and controller** for a collection of Student objects. It manages the list of students and exposes operations: add, remove, search, and display.

```
class StudentManager:
    def __init__(self):
        self.students = []   # empty list to hold Student objects

    def add_student(self, student):
        # Prevent duplicate roll numbers
        for existing_student in self.students:
            if existing_student.roll_no == student.roll_no:
                raise ValueError('Roll number already exists!')
        self.students.append(student)
        print(f'{student.name} added successfully!')

    def remove_student(self, roll_no):
        for student in self.students:
            if student.roll_no == roll_no:
                self.students.remove(student)
                print(f'Student removed with roll_no: {roll_no}')
                return
        print(f'Student with roll_no {roll_no} not found!')

    def search_student(self, roll_no):
```

```
        for student in self.students:
            if student.roll_no == roll_no:
                return student
        return None

    def display_students(self):
        print('Displaying all students below ->')
        for student in self.students:
            student.display_details()
            print('-' * 10)
```

■ Design Pattern Insight

This two-class design (Student + StudentManager) mirrors the Repository Pattern used in professional software: one class models the entity, another manages the collection. You will see this in Django ORM, SQLAlchemy, and data pipeline orchestrators.

4. Exception Handling

4.1 Why Exception Handling?

Real-world programs receive unpredictable input. Without exception handling, invalid data causes a program to crash entirely. Exception handling allows you to anticipate errors, respond gracefully, and keep the program running.

- Prevents unexpected program crashes.
- Provides meaningful error messages to users.
- Allows the program to recover or log issues.
- Essential in production data pipelines where failures must be caught and handled.

4.2 Using raise and ValueError

The **raise** statement is used to intentionally trigger an exception when a business rule is violated. **ValueError** is raised when a function receives an argument with an inappropriate value.

```
# Inside Student.__init__
if not name:
    raise ValueError('Name cannot be empty!')

# Inside StudentManager.add_student
for existing_student in self.students:
    if existing_student.roll_no == student.roll_no:
        raise ValueError('Roll number already exists!')
```

Catching exceptions:

```
try:
    manager.add_student(Student('', 5, 80))
except ValueError as e:
    print(f'Error: {e}')   # Output: Error: Name cannot be empty!
```

4.3 Common Exception Types

Exception	When to Use
ValueError	Wrong value type or invalid value (e.g., empty name)
TypeError	Wrong data type passed to a function
KeyError	Accessing a missing dictionary key
IndexError	Accessing a list index out of range

FileNotFoundError	File or path does not exist
Exception	Base class — catch-all for unexpected errors

5. Decorators

5.1 What is a Decorator?

A **decorator** is a design pattern in Python that allows you to modify or extend the behaviour of a function **without changing its source code**. A decorator is itself a function that takes a function as input and returns a new, enhanced function.

■ Mental Model

Think of a decorator as a wrapper. You have a gift (the original function). You wrap it in gift paper (the decorator). From the outside it looks the same, but now it has extra packaging around it.

- Decorators follow the **Open/Closed Principle**: open for extension, closed for modification.
- They enable cross-cutting concerns: logging, timing, authentication, caching.
- Widely used in frameworks like Flask (@app.route), Django (@login_required), and FastAPI.

5.2 Decorator Syntax & Mechanics

Python provides the **@decorator** syntactic sugar. The two forms below are equivalent:

```
# Without @ syntax (manual wrapping)
def greet():
    print('Hello!')

greet = decorator_function(greet)   # manually apply decorator
greet()                              # calls the wrapper

# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# With @ syntax (syntactic sugar)
@decorator_function
def greet():
    print('Hello!')

greet()   # same behaviour
```

■ Note

• The @ syntax is just syntactic sugar — Python replaces it with greet = decorator_function(greet) automatically.

5.3 Building a Logging Decorator

The logging decorator from class records when a function starts and finishes. This is a common pattern in production systems for monitoring and debugging.

```
def logger(func):
```

```
    def wrapper(*args, **kwargs):
        print(f'Starting {func.__name__}')
        result = func(*args, **kwargs)   # call the original function
        print(f'Finished {func.__name__}')
        return result   # always return the result!
    return wrapper
```

Line-by-line breakdown:

- **def logger(func):** — accepts the function to be decorated.
- **def wrapper(*args, **kwargs):** — inner function; *args and **kwargs allow it to wrap any function signature.
- **func.__name__** — accesses the original function's name for the log message.
- **result = func(*args, **kwargs)** — calls the original function, forwarding all arguments.
- **return result** — returns the original function's return value so nothing is lost.
- **return wrapper** — returns the wrapper; this replaces the original function.

5.4 *args and **kwargs in Decorators

Using ***args** and ****kwargs** makes a decorator **universal** — it can wrap any function regardless of how many positional or keyword arguments it takes.

```
# Without *args/**kwargs — only works for functions with no arguments
def wrapper():
    func()
# With *args/**kwargs — works for ANY function
def wrapper(*args, **kwargs):
    result = func(*args, **kwargs)
    return result
```

Applying the Logger to StudentManager:

```
class StudentManager:
    @logger
    def add_student(self, student):
        # ... implementation ...
        pass
    @logger
    def remove_student(self, roll_no):
        # ... implementation ...
        pass
# When add_student is called:
# Output:
```

```
# Starting add_student
# Vibhore added successfully!
# Finished add_student
```

6. Writing Maintainable Code

Maintainable code is code that is easy to understand, modify, test, and extend — even by someone who did not write it originally. In professional environments, code is read far more often than it is written.

- **Single Responsibility Principle (SRP)** — Each class/function does one thing. Student stores student data; StudentManager manages the collection.
- **Meaningful Names** — Use roll_no not r, add_student not func1. Names should communicate intent.
- **Input Validation** — Validate at the boundary — check inputs as soon as they enter the system.
- **Early Return** — In remove_student, return immediately after removing instead of using a flag variable.
- **PEP 8** — Follow Python's style guide: 4-space indentation, spaces around operators, snake_case names.
- **Comments** — Comment why, not what. Code already shows what it does; a comment explains the reasoning.
- **Avoid Magic Numbers** — Replace raw numbers with named constants: PASS_MARK = 60 instead of if marks >= 60.

7. Practical Example: Student Management System

The following is the complete, production-clean version of the Student Management System from class, combining all concepts: OOP, exception handling, and the logging decorator.

```
# ═══════════════════════════════════════════════════════════
# Logging Decorator
# ═══════════════════════════════════════════════════════════
def logger(func):
    def wrapper(*args, **kwargs):
        print(f'Starting {func.__name__}')
        result = func(*args, **kwargs)
        print(f'Finished {func.__name__}')
        return result
    return wrapper

# ═══════════════════════════════════════════════════════════
# Student Class
# ═══════════════════════════════════════════════════════════
class Student:
    def __init__(self, name, roll_no, marks):
        if not name:
            raise ValueError('Name cannot be empty!')
        self.name    = name
        self.roll_no = roll_no
        self.marks   = marks

    def calculate_grade(self):
        if self.marks >= 90:   return 'A'
        elif self.marks >= 75: return 'B'
        elif self.marks >= 60: return 'C'
        else:                  return 'D'

    def display_details(self):
        print(f'Name:   {self.name}')
        print(f'Roll:   {self.roll_no}')
        print(f'Marks:  {self.marks}')
        print(f'Grade:  {self.calculate_grade()}')

# ═══════════════════════════════════════════════════════════
# StudentManager Class
# ═══════════════════════════════════════════════════════════
class StudentManager:
```

```
    def __init__(self):
        self.students = []

    @logger
    def add_student(self, student):
        for existing in self.students:
            if existing.roll_no == student.roll_no:
                raise ValueError('Roll number already exists!')
        self.students.append(student)
        print(f'{student.name} added successfully!')

    @logger
    def remove_student(self, roll_no):
        for student in self.students:
            if student.roll_no == roll_no:
                self.students.remove(student)
                print(f'Student removed: roll_no={roll_no}')
                return
        print(f'Student with roll_no {roll_no} not found!')

    def search_student(self, roll_no):
        for student in self.students:
            if student.roll_no == roll_no:
                return student
        return None

    def display_students(self):
        print('All Students:')
        for student in self.students:
            student.display_details()
            print('-' * 20)

# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Usage
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
manager = StudentManager()

std1 = Student('Vibhore', 21, 99)
std2 = Student('Shachi',   1, 100)
std3 = Student('Moksh',    4,  10)

manager.add_student(std1)
manager.add_student(std2)
manager.add_student(std3)
manager.display_students()

manager.remove_student(4)
```

```
manager.display_students()
```

Expected Output:

```
Starting add_student
Vibhore added successfully!
Finished add_student
Starting add_student
Shachi added successfully!
Finished add_student
Starting add_student
Moksh added successfully!
Finished add_student
All Students:
Name:   Vibhore  Roll: 21  Marks: 99  Grade: A
--------------------
Name:   Shachi   Roll:  1  Marks: 100 Grade: A
--------------------
Name:   Moksh    Roll:  4  Marks: 10  Grade: D
--------------------
Starting remove_student
Student removed: roll_no=4
Finished remove_student
All Students:
Name:   Vibhore  Roll: 21  Marks: 99  Grade: A
--------------------
Name:   Shachi   Roll:  1  Marks: 100 Grade: A
--------------------
```

8. Real-World Data Engineering Perspective

OOP in Data Pipelines

In production data engineering, OOP is used extensively. Almost every major framework is built on classes: Apache Spark's DataFrame, Pandas' DataFrame, SQLAlchemy's Base, Airflow's DAG and BaseOperator — all are classes you instantiate and extend.

Concept	Data Engineering Equivalent
Student class	A schema class (e.g., a Pydantic model validating an event payload)
StudentManager class	A pipeline stage class that processes a batch of records
add_student validation	Schema validation / deduplication in an ingestion pipeline
Logger decorator	Observability middleware — log every ETL step start/end, duration, row count
ValueError on duplicate	Idempotency check — prevent duplicate records in a data lake

Decorators in Production Systems

- **Apache Airflow** — @dag and @task decorators define DAGs and tasks declaratively.
- **FastAPI** — @app.get('/endpoint') routes HTTP requests to handler functions.
- **Great Expectations** — decorators mark functions as data validation checks.
- **Custom ETL frameworks** — a @retry decorator automatically retries failed API calls.
- **Timing decorator** — wraps Spark transformations to measure and log execution time.

Exception Handling in ETL Pipelines

In production ETL, unhandled exceptions cause entire pipeline runs to fail. Professional pipelines always include structured exception handling:

```
def process_record(record):
    try:
        validated = validate_schema(record)
        transformed = apply_transformations(validated)
        load_to_warehouse(transformed)
    except ValueError as e:
        dead_letter_queue.append(record)   # quarantine bad records
        logging.error(f'Schema error: {e}')
```

```
    except ConnectionError as e:
        raise                                # propagate infra errors
```

■ Note

• In real pipelines, exceptions are categorised: data errors go to a dead-letter queue while infrastructure errors trigger alerts and retries.

9. Summary & Quick Revision Sheet

Key Concepts Summary

- **Class** — Blueprint for creating objects. Defined with the class keyword.
- **Object** — An instance of a class. Each object has its own attribute values.
- **__init__** — Constructor method — runs when an object is created.
- **self** — Reference to the current object instance inside a method.
- **Instance method** — A function defined inside a class that receives self.
- **raise** — Intentionally triggers an exception when a rule is violated.
- **ValueError** — Exception for invalid argument values.
- **Decorator** — A function that wraps another function to extend its behaviour.
- **wrapper** — The inner function inside a decorator — replaces the original.
- **@syntax** — Syntactic sugar: @deco above def is the same as func = deco(func).
- ***args/**kwargs** — Allows a decorator wrapper to accept any function signature.
- **func.__name__** — Accesses the original function's name string.

Syntax Cheat Sheet

```
# Class definition
class ClassName:
    def __init__(self, param1, param2):
        self.param1 = param1
        self.param2 = param2

    def method(self):
        return self.param1

# Instantiation
obj = ClassName(value1, value2)
obj.method()

# Raise exception
if not value:
    raise ValueError('Message')

# Catch exception
try:
    risky_operation()
except ValueError as e:
    print(e)
```

```python
# Decorator definition
def my_decorator(func):
    def wrapper(*args, **kwargs):
        # before logic
        result = func(*args, **kwargs)
        # after logic
        return result
    return wrapper

# Apply decorator
@my_decorator
def my_function():
    pass
```

10. General Interview Questions

A. Beginner Level

Q1. What is a class in Python?

A class is a blueprint or template for creating objects. It bundles related data (attributes) and behaviour (methods) together. You define a class using the class keyword.

Q2. What is the difference between a class and an object?

A class is the definition/template. An object is a specific instance created from that class. A class is like a cookie cutter; an object is the actual cookie.

Q3. What is __init__ and when is it called?

__init__ is the constructor method in Python. It is automatically called when a new object is created. It is used to initialise the object's attributes.

Q4. What is self in Python?

self refers to the current instance of the class. It is the first parameter of every instance method and is passed automatically by Python. It allows methods to access and modify the object's own attributes.

Q5. How do you create an object from a class?

By calling the class like a function: obj = ClassName(arguments). Python calls __init__ automatically.

Q6. What is an instance attribute?

An instance attribute is a variable that belongs to a specific object. It is defined using self.attribute_name = value inside __init__. Each object has its own copy.

Q7. What is an instance method?

An instance method is a function defined inside a class that takes self as its first parameter. It can access and modify the object's attributes.

Q8. What does raise do in Python?

raise triggers an exception intentionally. It is used to enforce business rules — for example, raising a ValueError when an invalid value is passed.

Q9. What is a ValueError?

ValueError is a built-in Python exception raised when a function receives an argument of the correct type but an inappropriate value (e.g., an empty string for a name field).

Q10. What is a decorator in Python?

A decorator is a function that takes another function as input and returns a modified version of it. It adds behaviour before and/or after the original function runs without changing the original

function's code.

Q11. What does the @ symbol mean in Python?

The @ symbol is syntactic sugar for applying a decorator. @decorator above def func is equivalent to func = decorator(func).

Q12. Why do we use *args and **kwargs in decorator wrappers?

To make the wrapper function universal — able to accept any combination of positional and keyword arguments, so the decorator can wrap any function regardless of its signature.

B. Intermediate Level

Q1. What is the Single Responsibility Principle and how is it applied in this session?

SRP states that each class should have one reason to change. In our example, Student is responsible only for a single student's data and behaviour. StudentManager is responsible only for managing the collection. This separation makes both classes easier to test, modify, and understand independently.

Q2. How does the logger decorator work step by step when add_student is called?

1. Python sees @logger above add_student and executes add_student = logger(add_student). 2. logger(func) runs, receives add_student as func, and returns wrapper. 3. add_student now points to wrapper. 4. When manager.add_student(std) is called, wrapper(manager, std) runs. 5. wrapper prints 'Starting add_student', calls the original func (add_student), prints 'Finished add_student', and returns the result.

Q3. What is the difference between raise and try/except?

raise is used to signal that an error has occurred (the producer side). try/except is used to handle an error that may be raised (the consumer side). They work together: you raise an exception where the error is detected and catch it where you can handle it meaningfully.

Q4. Can a decorator be applied to a method inside a class? How?

Yes. The decorator is placed above the method definition with @decorator_name. Since the decorator wrapper uses *args/**kwargs, it automatically handles the self parameter that Python passes as the first argument.

Q5. What happens if the wrapper function does not return result?

The decorated function effectively returns None, even if the original function returns a value. This is a common mistake. Always return result inside the wrapper.

Q6. How does duplicate roll number prevention work in StudentManager?

In add_student, we iterate over the existing students list and compare each student's roll_no with the new student's roll_no. If a match is found, a ValueError is raised and the new student is NOT appended. This is an O(n) linear scan.

Q7. What is the difference between a function and a method?

A method is a function that is defined inside a class and operates on an object's data via self. A function is a standalone callable not associated with a class.

Q8. How would you make the logger decorator also record execution time?

Import the time module. Record time.time() before calling func and after. Print or log the difference. Example: start = time.time() ... duration = time.time() - start ... print(f'Duration: {duration:.2f}s').

Q9. What is the output if you try to add a student with the same roll number twice?

The @logger decorator prints 'Starting add_student'. Then the inner loop finds the duplicate and raises ValueError. Python propagates the exception. If not caught, the program prints a traceback. The duplicate student is NOT added to the list.

Q10. Why is using return inside remove_student important?

After removing the student we return immediately, which exits the method. Without return, the loop would continue iterating over the (now modified) list, potentially causing issues. It also prevents the 'Student not found' message from printing after a successful removal.

11. Data Engineer Interview Preparation

The following questions focus on how OOP, exception handling, and decorator patterns are applied in real-world Data Engineering contexts — production pipelines, ETL systems, and scalable architectures.

Q1. How is OOP used in production data pipelines?

OOP is central to most data engineering frameworks. In Apache Spark, you work with DataFrame objects. In Apache Airflow, you define DAGs and Operators as class instances. In SQLAlchemy, your database tables are classes inheriting from Base. In practice, Data Engineers often create custom pipeline classes — e.g., an Extractor class, a Transformer class, and a Loader class — each responsible for one stage of the ETL process.

Follow-up: 'Can you give an example of a class hierarchy in a data pipeline?' → E.g., BaseConnector → PostgresConnector, S3Connector.

Q2. How would you apply exception handling in an ETL pipeline to ensure data quality?

In a production ETL pipeline, exceptions are categorised: Data exceptions (e.g., schema validation errors, null values in required fields) should be caught, logged, and the bad record sent to a dead-letter queue (DLQ). Infrastructure exceptions (e.g., database connection failures, S3 access denied) should be caught, logged with full context, and potentially trigger an alert and retry with exponential backoff.

The goal is to process as many good records as possible while quarantining bad ones, rather than failing the entire pipeline on one bad row.

Q3. What is a dead-letter queue and how does it relate to exception handling?

A dead-letter queue is a staging area (could be a database table, S3 bucket, or Kafka topic) where records that fail processing are stored for later inspection and reprocessing. In exception handling terms: instead of crashing on a ValueError from a malformed record, you catch the exception and write the record to the DLQ with the error message and timestamp.

This enables: 1) the pipeline to continue processing the remaining records, 2) engineers to review and fix bad data, 3) reprocessing of fixed records.

Q4. How would you design a logging decorator for a production data pipeline?

A production-grade logging decorator would: 1) Log function name, timestamp, and input parameters. 2) Catch and log exceptions with full traceback. 3) Log execution duration. 4) Optionally log record counts (rows processed). 5) Integrate with a centralised logging system (e.g., CloudWatch, Datadog, Splunk) rather than just printing to stdout.

Example: @pipeline_logger wraps every ETL function and sends structured JSON logs to the observability platform.

Q5. What is idempotency in data pipelines and how does it relate to duplicate prevention?

Idempotency means running a pipeline multiple times produces the same result as running it once — there are no duplicate records. Our StudentManager demonstrates this at a micro level: checking for duplicate roll_no before inserting prevents duplicates in the students list.

In production pipelines, idempotency is achieved with: UPSERT/MERGE operations (insert if not exists, update if exists), partition-based overwriting in data lakes, unique constraint checks, or deduplication keys.

Q6. How would you refactor a procedural ETL script into an OOP-based pipeline?

Step 1: Identify the nouns in the script (data sources, transformations, targets) — these become classes. Step 2: Identify the verbs (extract, transform, validate, load) — these become methods. Step 3: Group related state (connection strings, schemas, configs) into class attributes. Step 4: Use __init__ to initialise connections and configs. Step 5: Add exception handling at each stage. Step 6: Apply logging decorators to key methods.

This mirrors exactly what we did in class: refactoring from standalone functions to Student + StudentManager.

Q7. When would you use a class vs. a function in a data pipeline?

Use a function for: stateless, single-purpose transformations (e.g., parse_date(value), clean_phone_number(value)).

Use a class for: stateful operations where you need to maintain context across multiple calls (e.g., a DatabaseConnector that maintains an open connection, a BatchProcessor that tracks records processed and errors encountered). Classes are preferred when the component has multiple related methods that share state.

Q8. How do Airflow's @dag and @task decorators work? How are they similar to our logger decorator?

Airflow's @dag decorator converts a Python function into a DAG object. The @task decorator converts a function into a TaskFlow task with dependency management. Both use the same Python decorator mechanism we covered: they wrap the function, intercept its execution, and add framework-level behaviour (registering the function as a DAG/task, handling dependencies, managing XCom for data passing).

Our @logger decorator is the same pattern: it wraps a function and adds logging behaviour without modifying the function's source code.

Q9. How would you handle schema evolution in an OOP-based pipeline?

Define a base schema class with common fields. Create subclasses for each version: SchemaV1, SchemaV2. In the __init__ of each version, validate and transform the incoming data to the current schema. The pipeline calls a factory function that returns the correct schema class based on a version field in the data.

This uses the OOP principle of inheritance and polymorphism — the pipeline code only needs to call schema.validate() regardless of which version it is.

Q10. What are the performance implications of using OOP in Python for large-scale data processing?

Python OOP adds some overhead vs. plain functions due to attribute lookups and method resolution. For small-to-medium datasets this is negligible.

For large-scale data (millions of rows), the bottleneck is almost never OOP overhead — it is I/O, serialisation, and network latency. Using vectorised operations (Pandas, NumPy, Spark) is far more impactful.

Best practice: use OOP for pipeline orchestration, configuration, and connection management. Use vectorised libraries for the actual data transformations.

Q11. How would you implement a retry decorator for failed API calls in a data ingestion pipeline?

A retry decorator wraps the API call function. On exception, it catches the error, waits (with exponential backoff), and retries up to a maximum number of attempts. After max retries, it re-raises the exception.

Libraries like tenacity provide production-ready retry decorators. The pattern is: @retry(stop=stop_after_attempt(3), wait=wait_exponential(multiplier=1, min=2, max=10)).

This is a direct extension of the logging decorator pattern from class.

Q12. How would you make the StudentManager thread-safe for a concurrent data pipeline?

The students list is a shared mutable resource. In a concurrent pipeline (e.g., multiple threads ingesting data), two threads could simultaneously read the list, both see no duplicate, and both insert the same record — a race condition.

Solution: use a threading.Lock(). Acquire the lock before checking and inserting. Release it after. In Python asyncio pipelines, use asyncio.Lock() instead.

Q13. Explain how dependency injection and OOP relate to testable pipeline code.

Dependency injection means passing dependencies (database connections, file systems, API clients) into a class via __init__ rather than hard-coding them.

For StudentManager: instead of creating a database connection inside the class, pass it in: StudentManager(db_connection=conn). In tests, pass a mock connection instead of a real one.

This makes the pipeline code completely unit-testable without needing a real database or API.

Q14. What is the Open/Closed Principle and how do decorators exemplify it?

The Open/Closed Principle states: software should be open for extension but closed for modification. Decorators are a perfect example: we add logging, timing, or retry behaviour to add_student WITHOUT modifying the add_student function's source code. The function is closed for modification (we never touch it) but open for extension (we can stack multiple decorators).

In pipelines: add monitoring to legacy ETL functions by applying a decorator rather than refactoring potentially fragile old code.

Q15. How would you design a pipeline class that logs all method calls automatically?

Option 1: Manually apply @logger to every method.

Option 2: Use a metaclass or __init_subclass__ to automatically apply the logger decorator to all methods at class definition time. This is advanced OOP used in frameworks like SQLAlchemy.

Option 3: Use Python's __getattribute__ to intercept every method call. This is how some monitoring libraries (e.g., OpenTelemetry instrumentation) auto-instrument existing classes without changing their source code.

■ Final Tip for Interviews

When answering Data Engineering interview questions, always connect the concept to a real scenario: 'In my pipeline, I used a decorator to log every ETL step...' Concrete examples from this class are valid interview examples.